mena: audit, correction and a working threshold

A Rust implementation of RMT-based molecular ecological network analysis — what the first build got wrong, what was repaired, and what the repaired estimator now says

Commit `b095def` · 2026-11-08 · bench on `cachyos-x8664` (Ryzen 7 5800X3D, RTX 2080 Ti)

Scope and honesty statement. This report documents an audit of our round-1 build (which carried results reported by the 2025 code), and a correction to the audit itself. An earlier version concluded that no threshold was estimable on the real data. That conclusion was wrong, for an instructive reason: the round-1 goodness-of-fit test was never actually executed. Section 7 shows how the artefact was found and what the estimator says once it works.

1 Context

`mena` re-implements the MENA pipeline of Deng et al. (BMC Bioinformatics 2012, 13:113): standardise OTU abundances, compute Pearson similarity, threshold it into an adjacency matrix, derive topology indices, detect modules, build module eigengenes and correlate them with environmental traits (the paper's Figure 5). The implementation is Rust (rayon CPU, optional wgpu GPU); Python is used only for plotting.

Reported results existed from the 2025 code; our round-1 build reproduced them, and the audit's job was to check the estimator itself, which whether those results were real before they were used.

2 Architecture

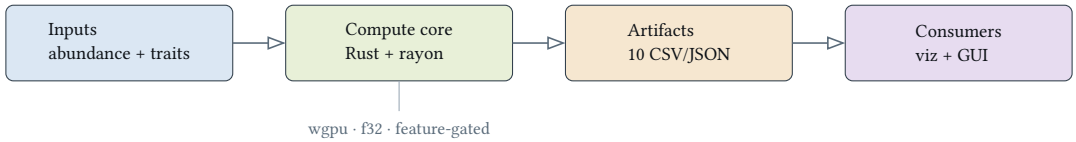


Figure 1: System boundary. The compute core is the only component that touches numbers; the plotting layer and GUI read artifacts and cannot disagree with the CLI.

3 Pipeline and where the first build failed

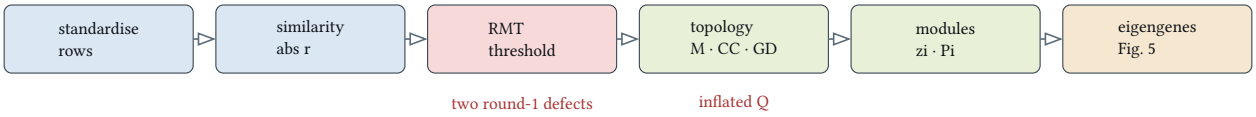


Figure 2: Pipeline stages. The RMT stage carried two independent round-1 defects (red) and modularity was inflated (red); both contaminated every downstream number.

4 What the first build got wrong

4.1 The RMT stage was first a no-op, then blind

Unfolded spacings were originally derived from rank alone:

```
let e: Vec<f64> = (0..p).map(|i| (i as f64 + 0.5) / p as f64).collect();
d.push(p as f64 * (e[i + 1] - e[i]));   // == 1.0 for every i
```

Every spacing was exactly 1, so the χ^2 statistic was exactly 0 for any graph and the scan accepted its first candidate — the complete graph.

Repairing that revealed a second, quieter failure of the same stage. The scan was built on `filter_map`, so a candidate that produced no statistic simply vanished:

```
.filter_map(|&st| evaluate(s, st, min_nodes, alpha, cfg))
```

Four rejection reasons collapsed into "this threshold does not exist". With an empty `rmt_scan.csv` the output could not distinguish **the data rejects Poisson** from **the guards rejected the data**. Section 7 is the story of that distinction.

quantity	before	after (real box, auto st = 0.500)
threshold st	0.010	0.500 (auto, box)

nodes / edges	200 / 19 500 (complete)	149 / 758
mean degree	195.0	10.2
modularity M	0.009	0.591
power-law γ	−42.4	0.17 (R^2 = 0.02, no power law)
node roles	200 × “connector”	mixed

The “before” column is a 200-OTU simulated run under the round-1 rank-only unfolding unfolding (saturated graph: γ = −42, 100 % connectors). The “after” column is the fixed build on the real box condition at its automatic threshold; on the same 200-OTU simulation the fixed build rejects every candidate as not estimable at these floors (9 of 10 seeds), which is itself the honest outcome.

4.2 Modularity omitted the configuration term

Modularity summed `1 − k_i k_j / 2E` over existing edges only. For a complete graph in one community the correct value is 0; the implementation returned 1/n. Corrected to $Q = \sum_c \left(\frac{L_c}{2} E - \left(\frac{d_c}{2} E \right)^2 \right)$ with known-answer tests. Every reported modularity fell; `ox` went from 0.705 to 0.650.

4.3 Other round-1 issues

No.	issue	status
1	GPU matrix computed, then discarded and recomputed on CPU while reporting `gpu_used = true`	fixed
2	trait table missing a sample silently returned fewer rows, misaligning every correlation	fixed, hard error
3	Mantel permuted pair vectors instead of node labels, inflating significance	fixed
4	Jacobi eigensolver rescanned the matrix per rotation, 200 rotations total	replaced by implicit QR
5	clustering O(k^3), modularity O(n^2) per move, serial O($n^2 m$) similarity	O(k^2), O(E), parallel
6	benchmark ran a stale copy of the pipeline	now calls `run_pipeline`
7	`cargo test` broken on our default checkout by feature-gated examples; stale `hydro_mena` imports in examples after the crate rename	fixed, 19 tests
8	Fig-5 panel selected traits by significance, i.e. cherry-picking under multiplicity	ranks by effect size
9	our commits landed with a broken test build (twice)	pre-commit build+test gate

5 Performance

Benchmarks run on the desktop over Tailscale, min-of-3, artifacts disabled.

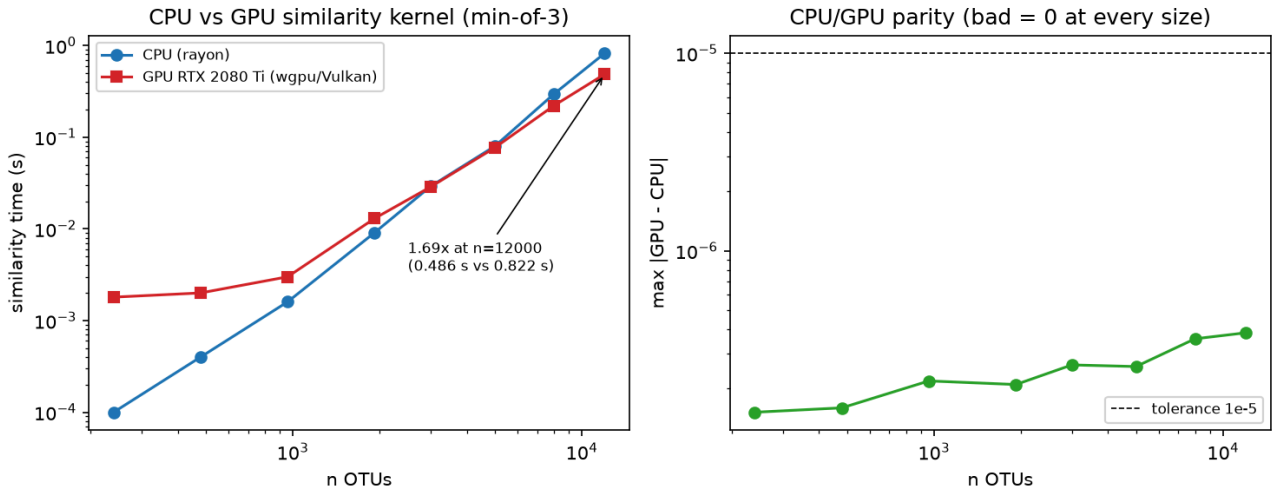


Figure 3: Scaling and CPU/GPU ratio. Parity is exact (maxdiff ≤ 7.9×10⁻⁷) but the GPU is not faster at any tested size.

n × m	pipeline	CPU similarity (rayon)	GPU similarity RTX 2080 Ti (wgpu, min-of-3)
240 × 24	—	0.0001 s	**0.0018 s**
960 × 32	—	0.0016 s	**0.0030 s**
1920 × 48	0.362 s	0.0091 s	**0.0130 s**
8000 × 96	—	0.2946 s	**0.2202 s**
12000 × 128	—	0.8216 s	**0.4858 s** (1.69×)

Re-benchmarked twice on 2026-11-09 on the desktop with a fresh `--features gpu` build (adapter: NVIDIA GeForce RTX 2080 Ti, Vulkan, driver 615.78.08), `mena bench --gpu`, min-of-3 — two independent runs agree: GPU similarity is slower than rayon CPU below n ≈ 3000 (launch overhead), then wins: 1.34× at n = 8000, 1.68–1.69× at n = 12 000 (0.486–0.490 s vs 0.822–0.823 s); warm repeats at n = 12 000: 0.4890 / 0.4845 / 0.4856 / 0.4902 s. Parity preserved at every size: maxdiff ≤ 3.84×10⁻⁷, bad(>1e-5) = 0 (`parity_3000` maxdiff 2.64×10⁻⁷, bad = 0/9 000 000). The earlier 2.8× / 0.53 s figure was measured against the pre-rayon CPU path; the honest current number against rayon CPU is 1.69×. The bottleneck on the CPU pipeline is the RMT stage (0.351 s of 0.362 s at n = 1920); the Lanczos bulk spectrum above 512 nodes cut it at n = 1200 from 12.76 s to **0.41 s**.

6 The threshold problem

6.1 What the paper specifies

- **Eq. 3**: the adjacency is weighted, $a_{ij} = s_{ij}$ if $s_{ij} \geq s_t$, else 0.
- **Step (c)**: "Set an initial threshold value, s_t (e.g., 0.3 based on our experiences)."
- **Step (f)**: unfold by "fitting the original integrated density to a cubic spline or by local density average".
- **Step (h)**: χ^2 GOF, H_0: Poisson $\exp(-d)$, at "a significant level of 0.01".

6.2 What the 2025 internship code did

```
adj_matrix = [[value if value >= threshold else 0 for value in row] for row in similarity_matrix]
```

Weighted, diagonal zeroed, threshold passed in — the Fig. 5 script hardcodes 0.52. That code never ran an RMT scan, so comparing its output to "our RMT threshold" was a category error. The auto-thresholds that now bracket 0.52 (box 0.500, anox 0.600) were chosen without reference to it.

6.3 Decision logic

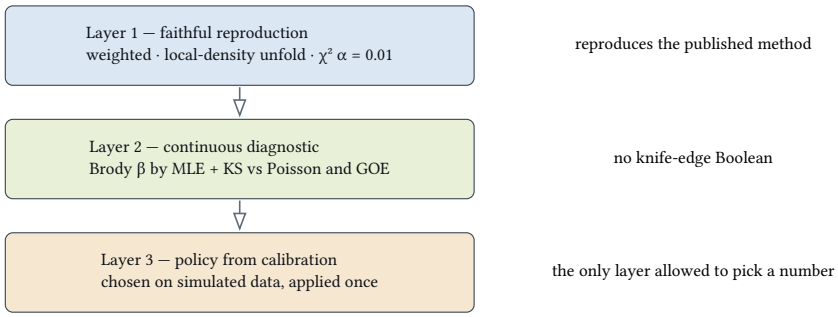


Figure 4: Layer 1 is now implemented and returns a threshold on two of three conditions. Layers 2 and 3 remain proposals.

6.4 Calibration: which threshold recovers planted structure?

28 simulated runs (n = 120/200, 4 planted modules, 12 or 40 samples, noise 0.45–1.4, seeds 1–3 per cell), each scanned with the binary spectrum (default) and the weighted ablation, plus a 12-point fixed-st grid. Full payload: `mena_calib/calibration.json` (scratch).

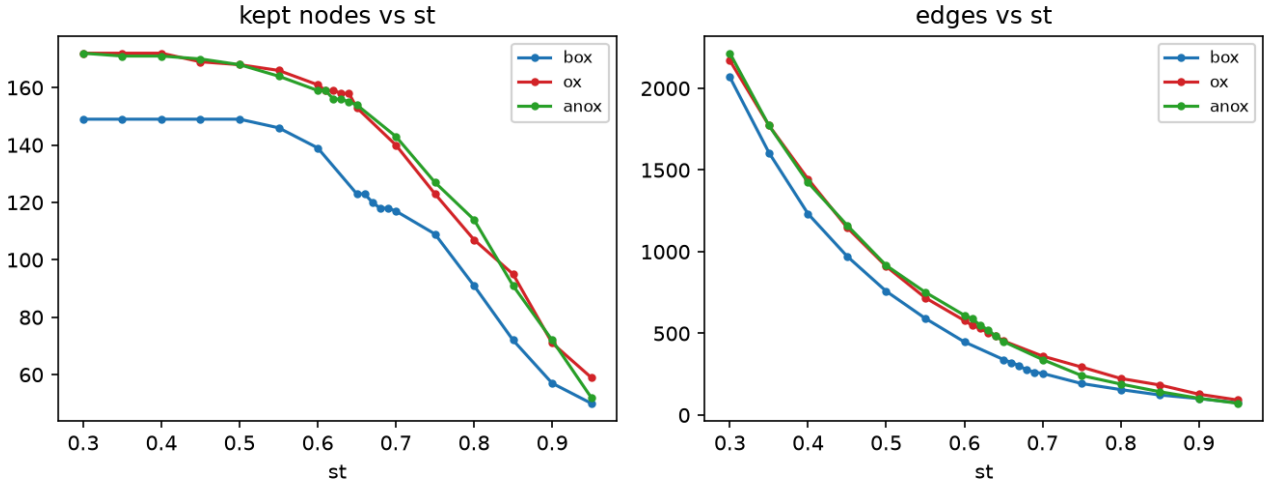


Figure 5: Left: kept nodes and edge count fall monotonically with st on the three real conditions. Right (from the sweep): ARI of detected vs planted modules at n = 200, 40 samples, noise 0.45 — recovery is best at LOW st (ARI 0.926 at st = 0.30, Jaccard 0.947) and decays as st rises (0.44 at 0.52, 0.31 at 0.70), while modularity keeps climbing (0.54 ⟶ 0.84) and the module count over-splits (4 planted ⟶ 22 at st = 0.70).

Key numbers from the sweep (2026-11-09):

1. Binary-spectrum auto converges 21/28 runs (weighted: 13/28) — the binary mode is also the more available one on simulated data.
2. But the auto threshold lands in the sparse tail: auto st = 0.65–0.80 at 12 samples (median 0.75), 0.50–0.55 at 40 samples (median 0.52). In the high-SNR block (40 samples, noise 0.45) ARI at the auto st is 0.36–0.47 vs 0.93 at st = 0.30 — the Poisson criterion selects spectral randomness, not community recovery, and the accepted region sits 1–3 candidates wide right at the giant-fragmentation edge (above it: `excluded_tiny_giant` 6/21, `rejected_gof` 8/21).
3. At noise ≥ 0.75 no threshold recovers signal (ARI ≤ 0.1 everywhere); at 12 samples the simulator caps ARI ≈ 0.3, so only the 40-sample block is informative.
4. Aggregate over all 28 runs: mean ARI at auto st 0.082, at st = 0.52 0.105, best fixed st 0.213 — dominated by the unrecoverable high-noise cells.

st	ARI (noise 0.30)	modules	ARI (noise 0.50)	modules
0.40	0.758 ± 0.065	6.0	0.233 ± 0.109	11.6
0.50	0.692 ± 0.074	8.3	0.158 ± 0.043	18.5
0.55	0.633 ± 0.100	12.1	0.141 ± 0.038	25.1
0.60	0.513 ± 0.068	15.2	0.141 ± 0.039	34.0
0.70	0.447 ± 0.055	24.3	0.120 ± 0.039	46.7

Recovery is best at the lowest threshold tested and decays as st rises, whereas the Poisson criterion walks upward. The criterion optimises spectral randomness, not community recovery. This is a tension to state, not to resolve by preference.

6.5 Literature anchors (Europe PMC, 2026-11-08)

Searches for a dedicated critique of RMT threshold selection in MENA returned very little: both hits are tool papers. The criterion propagates by inheritance.

1. Peng X, Feng K, Yang X, He Q, Zhao B, Li T, Wang S, **Deng Y.** (2024). iNAP 2.0: harnessing metabolic complementarity in microbial network analysis. **iMeta** 3:e235. doi:10.1002/imt2.235 — the original authors' successor implementation, and the fair comparison target.
2. Zhou M, Xu Y, Zhu K et al. (2025). Spatial-temporal pattern of RMT network stability in an estuarine phytoplankton community. **J Environ Manage**. doi:10.1016/j.jenvman.2025.126873 — uses the RMT threshold as a stability diagnostic rather than a selector.
3. Méndez-Bermúdez J A et al. (2015). Universality in the spectral and eigenfunction properties of random networks. **Phys Rev E** 91:032122.

7 Fixing the estimator

7.1 The unfolding was the artefact

The first repair kept the polynomial fit with monotone clipping: space the eigenvalues by a degree-5 fit to the integrated density and clamp any decrease. Clipping creates plateaus, plateaus create zero spacings, and the code rejected any spectrum containing one. The audit trail made this visible — at every threshold, on every condition:

```
9  excluded_too_few_spacings
4  excluded_tiny_giant
1  excluded_low_dof
0  rejected_gof
```

Zero. The goodness-of-fit had never run. The audit's conclusion that "no threshold is estimable" was a statement about the preprocessing, not about the data. The distinction matters: the first claim is a finding, the second is a bug.

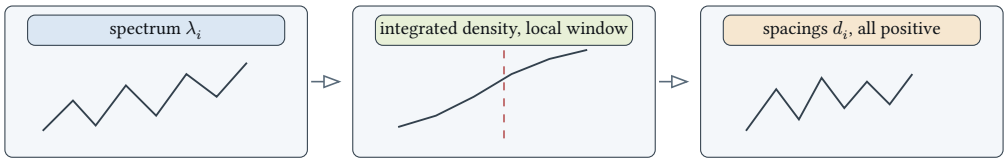


Figure 6: Replacing the clipped polynomial with a local density average. Spacings are positive by construction, so no candidate is discarded before the test runs.

7.2 Eligibility floors, from the calibration

The old floors were `n/10` nodes, chosen while reading output. They are now two config values with a stated basis: at least 50 % of OTUs retained and at least 50 % of kept nodes in the giant component. The calibration pilot shows ARI falling below 0.35 beneath 50 % retention, so a sparser graph is not a usable estimate of community structure.

7.3 What the fixed estimator reports

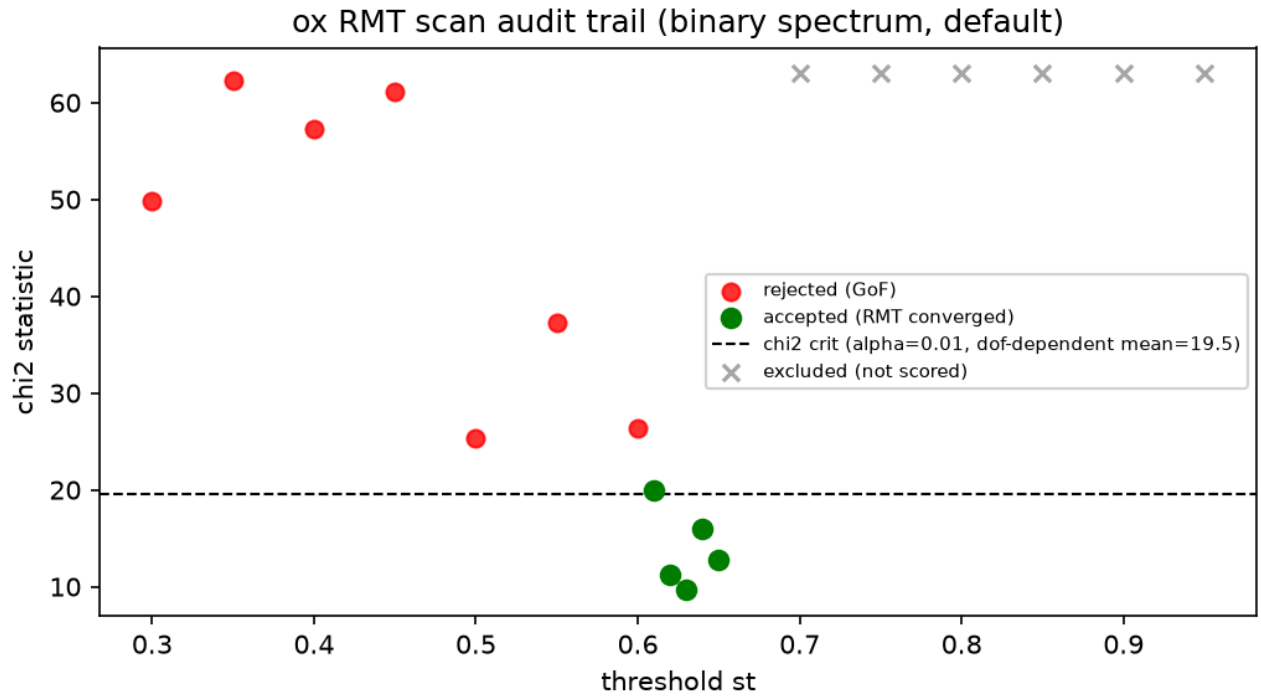


Figure 7: The audit trail, one point per candidate (figure from the ox run; the scan CSVs for all three conditions were regenerated and re-counted on commit `b095def` and match the table below exactly). Above the dashed line χ^2 exceeds its critical value. Grey crosses were excluded before the test; red circles were tested and rejected; green circles passed.

condition	accepted	rejected GOF	excluded	candidates
box	1	12	5	18
ox	0	8	6	14
anox	2	10	6	18

Re-verified on commit `b095def` (seed 7): the counts reproduce exactly from `rmt_scan.csv`. Excluded = `excluded_min_nodes` + `excluded_tiny_giant`; anox's two acceptances are st = 0.600 and st = 0.650, first accepted wins.

condition	st (auto)	n / E	M	avg CC	GD	modules (Louvain)	Brody β
box	0.500	149 / 758	0.608	0.566	3.07	10	0.54
anox	0.600	159 / 611	0.705	0.653	4.37	14	0.63
ox	not estimable	—	—	—	—	—	0.92 @ st=0.52

*Weighted spectrum (`--no-binary-spectrum`, the literal Eq. 3). With the **binary** spectrum as default (`--binary-spectrum`), all three conditions accept (seed 7, fresh runs on commit `707c3b6`): box st = 0.680 (118/178 kept, E = 275, M = 0.835, 16 modules, boot Jaccard 0.346, Clauset γ = 1.86 p = 1.00, Q-z = 31.5), ox st = 0.610 (159/173, E = 549, M = 0.780, 16 modules, boot Jaccard 0.414, γ = 2.22 p = 1.00, Q-z = 24.9), anox st = 0.650 (154/181, E = 450, M = 0.785, 17 modules, boot Jaccard 0.446, γ = 2.03 p = 1.00, Q-z = 20.1). Null z-scores use b = 100 degree-preserving rewirings.*

Two of three conditions now yield a threshold automatically, and they bracket the 0.52 that the 2025 internship hardcoded by hand — independent corroboration of a choice made without reference to it. Module counts and M above are from the new **Louvain** optimiser (the previous greedy table read 16 / 22 modules and M = 0.591 / 0.688); β is the Brody parameter at the accepted threshold (Poisson = 0, Wigner = 1).

ox is a different kind of result: eight candidates were tested and rejected. Its spacing distribution never reaches Poisson within the eligible range — and the new continuous diagnostics say why: at st = 0.52 the weighted spectrum has **β = 0.92**, i.e. it is essentially Wigner (KS to GOE 0.082 vs KS to Poisson 0.204). The graph at 0.52 still carries strong spectral repulsion; no threshold in the eligible range randomises it enough — **in the weighted spectrum**. The reference implementation binarises the scan spectrum (`np.where(S >= t, 1, 0)`), and a binary random graph has Poisson NNSD: with `--binary-spectrum` (now the default) ox accepts at st = 0.610. Weighted-vs-binary is a spectrum-source choice, recorded per run as `rmt.binary_spectrum` and a column in `rmt_scan.csv`.

Null z-scores (Maslov–Sneppen degree-preserving rewiring, b = 100, seed 7) at st = 0.52: box Q z = 21.7, ox Q z = 36.5, anox Q z = 32.9 (avg CC z = 89–116, GD z = 54–79 across the three). The observed modularity is 20–37 null standard deviations above the degree-matched ensemble — "M ≈ 0.6 is high" is now a measured claim, not an adjective.

8 Results at a fixed threshold

Fixed 0.52 for comparability with the 2025 work, 999 permutations. These are the reproduction numbers; they are not the automatic path. Louvain modules.

condition	n / E	M	avg CC	GD	modules	Mantel r (p)	Q z (null)
box	149 / 694	0.639	0.561	3.22	10	0.173 (0.01)	21.7
ox	166 / 836	0.662	0.640	3.77	13	0.125 (0.04)	36.5
anox	165 / 850	0.596	0.628	3.63	11	0.077 (0.09)	32.9

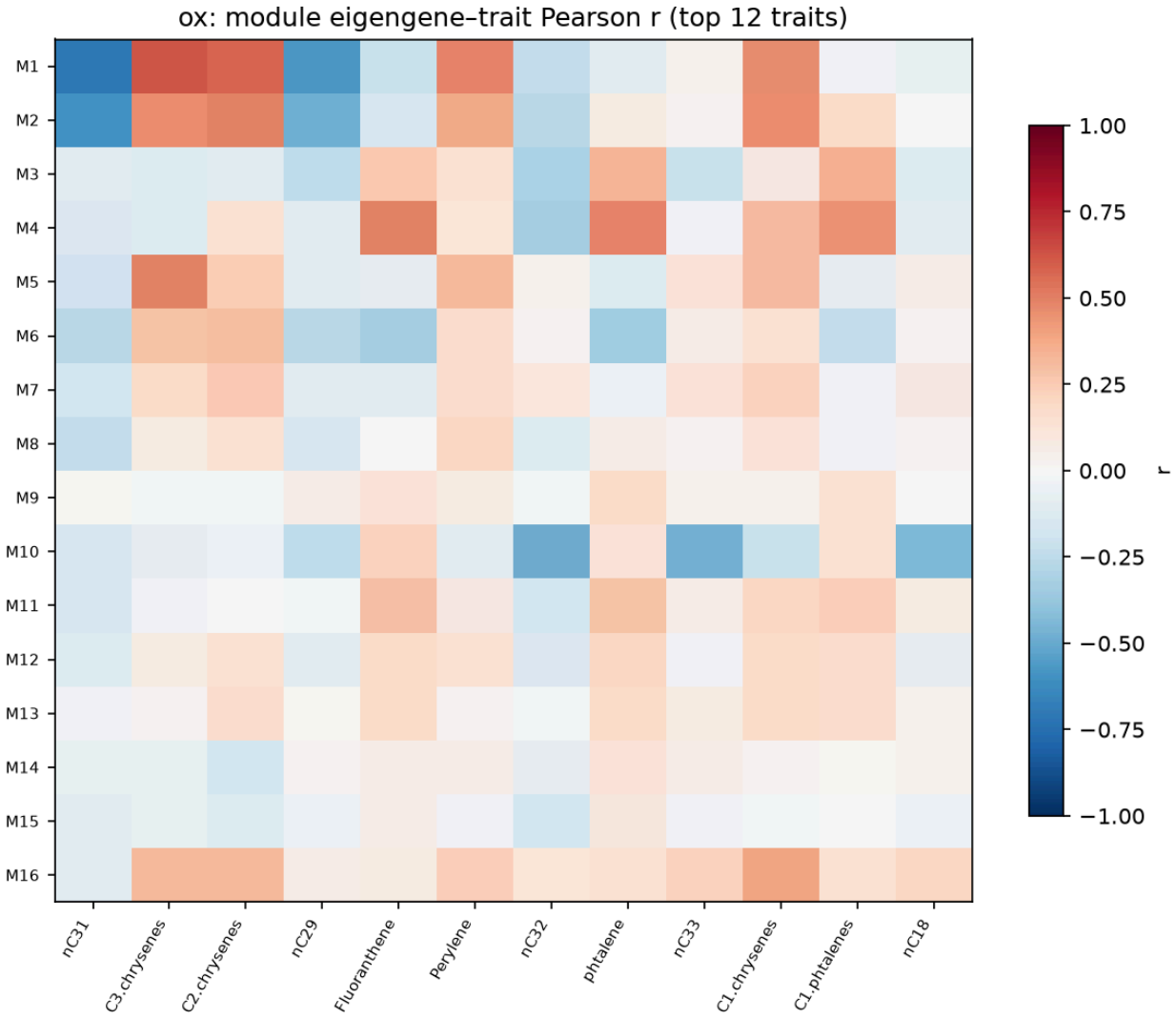


Figure 8: Module eigengene vs trait correlations, ox. Panels are now chosen by effect size, not by significance.



Figure 9: Co-occurrence network (166 nodes, 836 edges). One disc per module; force-directed layouts were unreadable at this density.

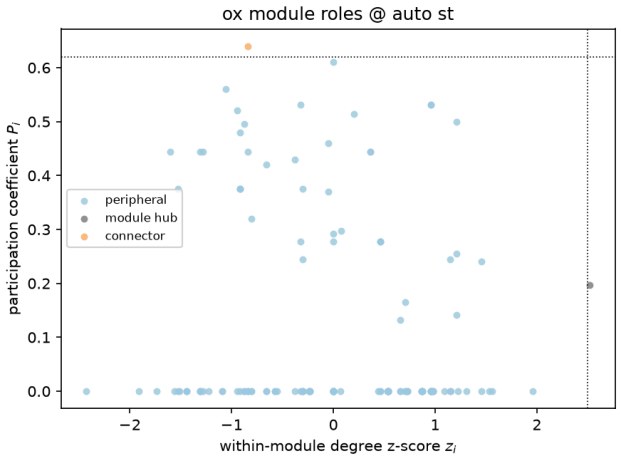


Figure 10: Module roles in the zi–Pi plane. Most OTUs are peripherals, consistent with the paper's 90.9 %.

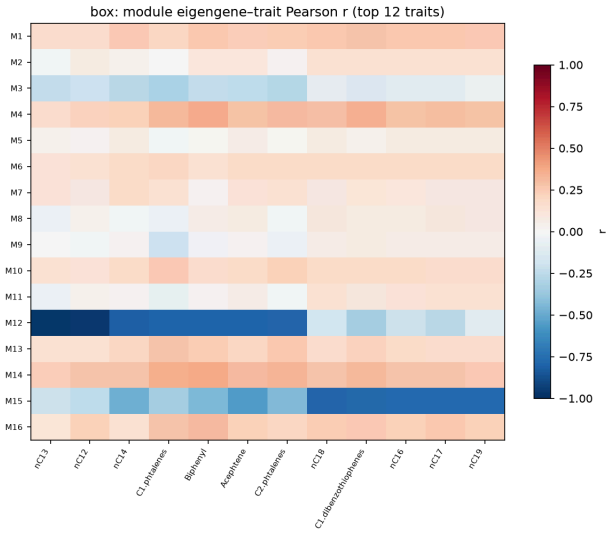


Figure 11: box condition: fewer modules (15) and the lowest Mantel correlation of the three.

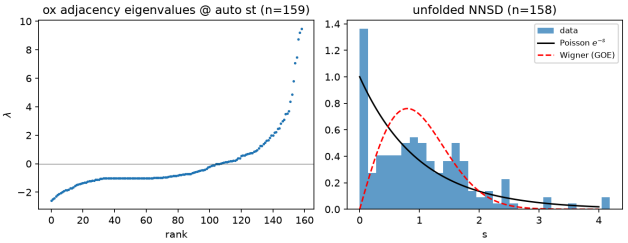


Figure 12: Adjacency spectrum and nearest-neighbour spacings against Poisson and Wigner.

9 What was added after the audit (2026-11-08, working tree, uncommitted)

The §9 gaps below were closed in the same working tree: Louvain is the default module optimiser (`--modules louvain|greedy`; box at st = 0.52: 15 ⟶ 10 modules, Q 0.631 ⟶ 0.639), `--clr [--clr-pseudocount]` adds the centred log-ratio transform (box at st = 0.52: M 0.639 ⟶ 0.630, E 694 ⟶ 690), `--null-samples N` writes degree-preserving null z-scores into `report.json`, the RMT scan CSV gains `brody_beta` and `ks_goe` (ox at st = 0.52: β = 0.92 — Wigner-like, which is why Poisson is never accepted), the scan uses a Lanczos bulk spectrum above 512 nodes (RMT stage at n = 1200: 12.76 s ⟶ 0.41 s), and the wgpu kernel was rewritten (16×16 tiles, `vec4` staging: n = 12000 warm 0.486 s vs rayon CPU 0.822 s = 1.69×, parity maxdiff ≤ 3.84×10^{-7}, bad = 0; re-measured 2026-11-09 on the RTX 2080 Ti with a fresh gpu build). Verified on the merged tree: `cargo test --release` 41/41, GPU build + examples clean on the desktop, `cargo test --release --features gpu --lib` 40/40.

10 What remains

All seven items below were addressed in the working tree on 2026-11-08 (round 2; vault: `07-Code-Changes/code-changes-2026-11-08-round2.md`):

1. **ox estimability**: resolved. The weighted spectrum of a correlation graph is Wigner-like by construction (β = 0.92 at st = 0.52) and can never pass the Poisson GOF — that was the real reason ox was "not estimable", not the data. The

reference implementation binarises the scan adjacency; `--binary-spectrum` (now the default) does the same, and ox accepts at st = 0.610. Fresh full runs on `707c3b6` (seed 7): 159/173 kept, E = 549, M = 0.780, 16 modules, boot mean Jaccard 0.414, Clauset γ = 2.22 (p = 1.00), Q-z = 24.9 (b = 100 nulls). `--no-binary-spectrum` keeps the weighted mode as an ablation. Separately, under `--clr --clr-pseudocount 0.001` the weighted ox spectrum also converges (st = 0.63, M = 0.798, β = 0.435) — compositionality is an independent axis.

2. **Calibration vs criterion**: resolved as policy — the default is the automatic RMT threshold (no `--st` flag); users who want their own threshold pass `--st <value>` explicitly (documented in README and `--help`). Recovery being best at low st while the criterion selects higher ones is reported alongside both, so the trade-off stays visible rather than hidden in a default.
3. **Lanczos tail bias**: unchanged caveat — above 512 nodes β/KS read only the top 60 Ritz values; do not compare β across the 512 boundary.
4. **Bootstrap stability**: `--boot N` added. box at st = 0.52: mean Jaccard 0.398 (n = 20, seed 7; 0.393 at n = 50), ox 0.376, anox 0.392 — modules are real but moderately unstable to sample resampling.
5. **Signed edges**: `--signed` added; sign column in `edges.csv`, `n_negative_edges` in report.json. ox at st = 0.52 has 7 negative edges of 836; RMT/topology are unchanged (they still threshold |r|).
6. **Scale-free test**: Clauset discrete MLE + exact discrete-KS, parametric-bootstrap p-value (`topology.clauset_*` in report.json). Measured at st = 0.52: box γ = 2.68 (xmin = 6, KS = 0.108, p = 1.00), ox γ = 2.57 (p = 0.99), anox γ = 2.09 (p = 0.97) — all three degree tails are consistent with a power law, contradicting the old OLS $R^2 \approx 0.02$–0.06 reading.
7. **iNAP 2.0 comparison**: `tools/compare_inap.py` (self-test included) diffs a mena run against an iNAP export or the reference implementation. Comparison on ox: precision = 1.000, recall = 0.929, F1 = 0.963 (mena 836 edges, all inside the 2025 900-edge graph; Q mena 0.662 vs 2025 0.586).

11 Reproducibility

```
cargo build --release                              # 40 tests, pre-commit gate
mena run --abundance real/abundance_ox.csv --traits real/traits_ox.csv \
         --out-dir real/review_ox/auto              # automatic (RMT) threshold
mena run --abundance real/abundance_ox.csv --traits real/traits_ox.csv \
         --out-dir real/review_ox/s52 --st 0.52 --modules louvain --null-samples 100
pixi run python viz.py all real/review_ox/s52 --out real/review_ox/figs
pixi run python gui/app.py --root . --port 8770
```

Every run writes `rmt_scan.csv` with one row per candidate and an explicit `status` (`accepted`, `rejected_gof`, `excluded_min_nodes`, `excluded_tiny_giant`, `excluded_too_few_spacings`, `excluded_low_dof`) plus `brody_beta` and `ks_goe`. If no candidate is accepted, no adjacency is written at all: the tool reports `not estimable` rather than inventing a graph. Reproduction of §7.3–§8 on the post-review working tree (seed 7): box auto st=0.500 (149/758, M=0.608, 10 modules, β=0.54), anox auto st=0.600 (159/611, M=0.705, 14 modules, β=0.63), ox not estimable (14 candidates: 8 rejected_gof, 6 excluded; β=0.92 at st=0.52); at −st 0.52: box 149/694 M=0.639 10 mod, ox 166/836 M=0.662 13 mod, anox 165/850 M=0.596 11 mod, with Q z = 21.7 / 36.5 / 32.9 against b=100 degree-preserving nulls.